TITRE PROFESSIONNEL • RNCP37674

Dossier de projet

L’Aiguille des Volcans

Site web de broderie et de personnalisation textile

Développeur web et web mobile
Niveau 5



Projet Symfony et MariaDB

Aureo Ruffier

L’Institut de la PME
Formation du 4 mai au 14 août 2026
Soutenance prévue le 29 octobre 2026

Réalisation individuelle
Version documentaire du 11 octobre 2026
État du code étudié b406ce8

Document de travail à actualiser après les correctifs et la recette finale.

Sommaire

01 Présentation du projet 4

02 Correspondance avec le référentiel 5

03 Expression des besoins 6

04 Acteurs et règles de gestion 7

05 Organisation et traçabilité du travail 8

06 Choix des technologies 9

07 Environnement de développement 10

08 Architecture de l'application 11

09 Parcours et enchaînement des écrans 12

10 Maquettage web et mobile 13

11 Identité graphique et hiérarchie visuelle 14

12 Réalisation des interfaces statiques 15

13 Adaptation aux petits écrans 16

14 Interactions JavaScript 17

15 Catalogue et navigation par catégorie 18

16 Modélisation conceptuelle des données 19

17 Modèle physique et dictionnaire des données 20

18 Migrations et données de démonstration 21

Sommaire et figures

19 Accès aux données avec Doctrine 22

20 Conception du formulaire de devis 23

21 Traitement serveur de la demande 24

22 Suivi administratif des devis 25

23 Avis et modération 26

24 Authentification et autorisations 27

25 Administration du catalogue et des comptes 28

26 Sécurité des données entrantes et des fichiers 29

27 Accessibilité et données personnelles 30

28 Stratégie de vérification 31

29 Jeux d'essai et résultats obtenus 32

30 Analyse des anomalies et traitement prévu 33

31 Procédure de préparation et de livraison 34

32 Site publié et mode maintenance 35

33 Sauvegarde et continuité de service 36

34 Veille technique et sécurité 37

35 Bilan et perspectives 38

36 Sources et pièces de preuve 39

Annexes : A Modèle physique p. 40 ; B Maquette web p. 41 ; C Extraits du formulaire p. 42 ; D Maquette mobile p. 43.

Figures 1 à 9 : architecture, parcours, wireframes, maquette mobile, accueil, rendu étroit, galerie, modèle conceptuel et séquence du devis.

01 Présentation du projet

L'Aiguille des Volcans est une application web de présentation et de gestion destinée à une activité artisanale de broderie et de personnalisation textile. Elle permet de découvrir les prestations, de consulter des réalisations, de transmettre une demande de devis et de publier un avis après modération. Le projet répond à un besoin de visibilité et de prise de contact, sans paiement ni commande en ligne.

Je présente ce projet dans le cadre du titre professionnel Développeur web et web mobile, RNCP37674, niveau 5. Je l'ai développé seul pendant ma formation à L'Institut de la PME, suivie du 4 mai au 14 août 2026. La soutenance est prévue le 29 octobre 2026. Le dossier décrit la version du dépôt examinée le 11 octobre 2026, dont la référence Git est b406ce8. Des évolutions postérieures à la période de formation apparaissent dans les fichiers et les migrations.

L'application associe Symfony et PHP pour le traitement serveur, Twig pour les vues, Bootstrap et du CSS personnalisé pour la présentation, JavaScript pour les interactions et MariaDB pour les données. Un espace d'administration donne accès aux produits, catégories, utilisateurs, avis et demandes de devis.

État présenté au jury

Le domaine https://www.laiguilledesvolcans.fr/ est publié. À la date de préparation du dossier, il affiche une page de maintenance au public. Selon les informations que j'ai communiquées pour ce dossier, un administrateur peut se connecter par /login et accéder au site complet. La configuration de l'hébergeur et une recette de production ne font pas partie des preuves disponibles ici.

L'analyse distingue les fonctions présentes dans le code, les contrôles effectivement exécutés et les améliorations restant à réaliser. Les captures récentes proviennent de réponses HTML du noyau Symfony local, rendues avec les ressources locales. Elles ne constituent pas des captures de la production ni une recette complète sur téléphone physique.

Le dossier suit le cheminement du besoin jusqu'au déploiement. Les limites constatées sont expliquées afin de permettre une démonstration technique précise et de préparer les dernières corrections avant l'examen.

Sources : cahier_des_charges.md ; dépôt b406ce8 ; présentation fournie ; informations du candidat du 11 octobre 2026.

02 Correspondance avec le référentiel

Le référentiel retenu est le REAC DWWM TP-01280, millésime 04, mis à jour le 24 mai 2023, complété par le référentiel d'évaluation de la même version. Le fichier nommé EDUCENTRE_RNCP31114_REAC contient bien cette version de 2023. Son nom de fichier ne suffit donc pas à identifier la version du titre. La fiche officielle RNCP37674 précise les deux blocs, front-end sécurisé et back-end sécurisé.

Le référentiel d'évaluation prévoit entre 30 et 50 pages, illustrations comprises, hors couverture, sommaire et annexes. La structure proposée initialement est pertinente, mais elle doit rendre les preuves de compétences faciles à retrouver et préciser les essais, les choix techniques et la veille de sécurité.

Compétence	Preuves présentées	État de couverture
CP1 Installer et configurer l'environnement	Docker, dépendances, Git et contrôles ; chap. 5 à 7	Mise en œuvre ; CI non établie
CP2 Maquetter les interfaces	Parcours, wireframes, maquettes web et mobile ; chap. 9 à 11	Supports présents ; validation client à tracer
CP3 Réaliser les interfaces statiques	Twig, HTML, CSS et responsive ; chap. 12 et 13	Mise en œuvre ; accessibilité à compléter
CP4 Développer les interfaces dynamiques	Thème, aperçu et lightbox ; chap. 14 et 15	JavaScript présent
CP5 Mettre en place une base relationnelle	MCD, modèle physique, migrations ; chap. 16 à 18	Mappage et schéma contrôlés
CP6 Développer l'accès aux données	Repositories et requêtes liées ; chap. 19	Accès SQL avec MariaDB et Doctrine
CP7 Développer les composants métier	Devis, avis, comptes et CRUD ; chap. 20 à 25	Limites identifiées
CP8 Documenter le déploiement	Procédure et exploitation ; chap. 31 à 33	Preuves de production partielles

La matrice présente les compétences mobilisées dans le périmètre du projet, avec des intitulés abrégés. L'accès aux données est illustré par les repositories Doctrine et les requêtes exécutées sur MariaDB. Les exemples relient la sélection, l'enregistrement et la modification des données aux fonctionnalités du catalogue, des devis et des avis.

Sources : REAC 2023, vue synoptique p. 8 et fiches CP ; RE 2023 p. 5-6 ; France compétences RNCP37674.

03 Expression des besoins

Le cahier des charges définit un site vitrine interactif pour une artisane brodeuse. Les visiteurs doivent comprendre les prestations et disposer d'un moyen simple de décrire leur projet. L'administratrice doit pouvoir actualiser le catalogue et traiter les demandes sans modifier directement les fichiers du site.

Le public comprend les particuliers, les entreprises, les associations et les collectivités. Ces catégories sont reprises dans le formulaire de devis. Une association peut, par exemple, demander la personnalisation de plusieurs polos, préciser qui fournit les textiles et joindre un logo. Une personne recherchant un cadeau peut commencer par la galerie et utiliser les réalisations comme support de discussion.

Périmètre fonctionnel

Le catalogue présente des produits ou réalisations illustrés, associés à des catégories. Les prix affichés sont indicatifs : ils ne constituent pas un calcul automatique de devis. La demande enregistre des coordonnées, une description et des paramètres de personnalisation. L'accord commercial, la préparation des fichiers de broderie, la fabrication et la facturation restent en dehors de l'application.

L'inscription et la connexion permettent de déposer un avis. Le visiteur peut demander un devis sans créer de compte. Cette différence réduit les étapes nécessaires à une première prise de contact, tout en associant les avis à un utilisateur identifié. L'administration couvre les opérations de création, lecture, modification et suppression des principales données.

Objectifs de qualité

Le parcours doit rester compréhensible sur ordinateur comme sur mobile. Les informations doivent être cohérentes entre le catalogue et les fiches, les actions sensibles doivent être réservées à l'administration et les erreurs de formulaire doivent être explicites. La confidentialité des coordonnées et des pièces jointes doit être prise en compte dès leur collecte.

Le cahier des charges mentionne une version française et anglaise. Les interfaces examinées sont principalement en français et aucun parcours bilingue complet n'est démontré. La traduction est donc présentée comme une évolution. De même, le dépôt contient une méthode de recherche par mot-clé, mais pas un parcours de recherche effectivement relié au contrôleur public examiné.

Sources : output/docs/cahier_des_charges.md ; QuoteRequestType ; ProductRepository ; templates publics.

04 Acteurs et règles de gestion

Trois profils structurent l'application. Le visiteur consulte les contenus publics et envoie une demande de devis. L'utilisateur authentifié peut également soumettre un avis. L'administrateur gère le catalogue, les comptes, les demandes et la modération. Les contrôles d'accès doivent être exercés côté serveur, car masquer un lien dans une vue ne protège pas sa route.

Action	Visiteur	Utilisateur	Administrateur
Consulter le catalogue public	Oui hors maintenance	Oui hors maintenance	Oui
Envoyer un devis	Oui hors maintenance	Oui hors maintenance	Oui
Déposer un avis	Connexion demandée	Oui	Oui
Gérer produits et catégories	Non	Non	Oui
Modérer les avis et traiter les devis	Non	Non	Oui
Accéder au site pendant la maintenance	Connexion seulement	Non pour les autres pages	Oui

Règles déterminantes

Un avis est créé avec le statut pending. Seuls les avis approved sont sélectionnés pour les listes publiques. La note proposée est comprise entre 1 et 5. L'avis peut concerner un produit ou l'atelier en général ; son utilisateur est obligatoire dans le modèle de données.

Une demande de devis reçoit à sa création le statut new. Les autres valeurs autorisées sont processing et finished. Le setter de l'entité refuse une valeur inconnue. Cette liste limite les états possibles, mais elle ne définit pas une machine à états : le code n'interdit pas toutes les transitions de retour entre valeurs autorisées.

Un produit possède un indicateur isActive. La galerie générale, la fiche produit et la requête spécialisée par catégorie filtrent les produits actifs. En revanche, la vue de catégorie parcourt directement category.products sans appliquer ce filtre. Cette divergence peut exposer un produit désactivé dans une liste, alors que sa fiche publique renvoie une erreur 404.

Le rôle ROLE_USER est ajouté par getRoles à chaque utilisateur. Le préfixe des noms de routes admin_ est protégé par security.yaml. La cohérence de ce nommage est donc une condition importante lors de l'ajout de nouvelles fonctions d'administration.

Sources : security.yaml ; ReviewController ; QuoteRequest ; ProductController ; front/category/show.html.twig.

05 Organisation et traçabilité du travail

J'ai réalisé seul le développement. L'environnement humain comprend le contexte de formation à L'Institut de la PME et le besoin métier de l'atelier. Aucun rôle de chef de projet, de développeur supplémentaire ou de prestataire technique n'est attribué sans preuve. La présentation commencée mentionne comme difficultés la compréhension de Symfony avec Docker, les relations Doctrine, les échanges entre contrôleurs et vues et l'intégration responsive.

La période de formation communiquée s'étend du 4 mai au 14 août 2026. Les migrations présentes commencent le 29 juillet et se poursuivent en septembre. Elles montrent une évolution technique du schéma ; elles ne permettent pas de reconstituer à elles seules le calendrier complet du travail ni le temps passé sur chaque fonctionnalité.

Organisation reconstituée à partir des livrables

Les supports disponibles permettent de distinguer l'expression du besoin, les maquettes, la modélisation, le développement des parcours publics, le back-office et la préparation de l'exploitation. Cet ordre est un classement des preuves du projet. Il ne constitue pas un planning initial signé ni la preuve de sprints Scrum, de réunions régulières ou d'une recette client formalisée.

Git conserve les versions du code. Le dernier historique examiné contient notamment des évolutions liées à la maintenance, aux photos, aux formulaires d'administration et aux interactions JavaScript. Des messages de commit courts rendent l'historique moins facile à interpréter. Pour les prochaines évolutions, des messages décrivant le problème résolu et un lien vers une fiche de tâche amélioreraient la traçabilité.

Gestion des changements

Une modification devrait être décrite par son besoin, son impact sur les données et ses critères d'acceptation. Pour un ajout de champ de devis, cela signifie modifier le formulaire, le modèle, la migration, l'affichage administratif et les essais correspondants. Une simple capture de la nouvelle interface ne prouve pas la cohérence de l'ensemble.

Le dossier présente l'état disponible au 11 octobre 2026. Les supports graphiques et la présentation peuvent décrire des étapes antérieures. Lorsqu'ils divergent du code actuel, la différence est signalée. Les dates des échanges client et la validation des maquettes restent des pièces utiles à conserver pour la soutenance.

Sources : historique Git ; migrations ; présentation fournie, diapositives 21 et 22 ; informations du candidat.

06 Choix des technologies

Symfony organise le traitement des requêtes, les formulaires, la validation, la sécurité et l'injection de dépendances. Ce cadre est adapté à un site dont les données et les règles métier sont centralisées côté serveur. Twig produit directement le HTML ; JavaScript enrichit les interactions sans imposer une application monopage ni une API séparée.

Élément	Version ou contrainte relevée	Rôle
PHP	Minimum 8.2 dans composer.json	Exécution serveur
Symfony	Branche 7.4 demandée	Routes, services, formulaires et sécurité
Doctrine ORM	Contrainte ^3.6.8	Mappage objet relationnel
MariaDB	Image 10.11.11	Stockage relationnel
Twig	Contrainte ^2.12 ou ^3.0	Gabarits HTML
Bootstrap	Contrainte ^5.3.8	Grille et composants
Webpack Encore	Contrainte ^7.2.0	Compilation des ressources
Node.js	Image 22 dans Compose	Outils du front-end
Docker Compose et Caddy	Caddy 2.11 dans le Dockerfile	Environnement HTTP local

Les contraintes déclarées ne sont pas toutes des versions réellement exécutées. Pendant les vérifications, PHP dans le conteneur a annoncé 8.5.9, alors que le Dockerfile fourni commence par php:8.2-fpm. Cette divergence doit être résolue ou documentée pour garantir la reproductibilité. Le dossier ne remplace pas cette mesure par la version théorique du fichier.

Doctrine évite de répéter le code de connexion et fournit un langage de requêtes adapté aux entités. MariaDB convient aux relations entre produits, catégories, utilisateurs et avis. Bootstrap accélère l'intégration de la grille et des formulaires ; le CSS personnalisé conserve l'identité visuelle de l'atelier.

Les fichiers composer.lock et package-lock.json fixent les dépendances résolues. Le projet contient aussi AssetMapper et importmap.php, mais les layouts examinés chargent les entrées Encore. Il faut éviter de présenter deux chaînes de ressources comme simultanément nécessaires si une seule est réellement utilisée par ces pages.

Sources : composer.json et lock ; package.json et lock ; Dockerfile ; compose.yaml ; layouts ; vérification PHP.

07 Environnement de développement

Le projet est disponible dans un espace WSL Ubuntu utilisé depuis Windows. Docker Compose déclare cinq services : php, caddy, node, mariadb et phpmyadmin. Les fichiers du projet sont montés dans les conteneurs afin que les modifications soient immédiatement accessibles. Un volume dédié conserve les données MariaDB indépendamment du cycle de vie du conteneur.

Caddy sert le répertoire public et transmet les requêtes PHP au service php sur le port 9000. Le point d'accès local est https://localhost:8443. La directive tls internal utilise une autorité locale ; ce paramétrage ne démontre pas la présence d'un certificat de production valide pour le domaine public.

Le service node installe les dépendances puis lance le mode watch. Les entrées scripts et styles sont définies dans webpack.config.js. Le build copie également les images vers public/build/images. Le mode production active les noms de fichiers versionnés et désactive les source maps prévues pour le développement.

Reproductibilité et outils de diagnostic

Les contrôles de syntaxe Twig et YAML et la validation du schéma Doctrine permettent de détecter rapidement des incohérences. PHPUnit, BrowserKit et CssSelector sont installés pour construire des tests. Leur présence ne signifie pas qu'une suite de tests existe : l'exécution actuelle de PHPUnit ne lance aucun test.

Le dépôt contient un Makefile qui facilite le démarrage et les opérations de base. Certaines cibles suppriment et recréent la base, puis importent des données. Elles doivent rester limitées à un environnement jetable. Une procédure de déploiement ne doit pas recopier ces commandes de réinitialisation vers un environnement contenant des demandes réelles.

Les paramètres sensibles se configurent par environnement. Le dossier ne reproduit ni secret d'application, ni identifiant SMTP, ni mot de passe réel. Pour rendre l'installation transmissible, une liste des variables attendues et une procédure de création d'un compte administrateur doivent accompagner les instructions techniques, sans contenir leurs valeurs confidentielles.

La configuration de développement expose aussi MariaDB et phpMyAdmin sur des ports de l'hôte. Une exploitation publique exige une configuration distincte et un accès restreint à ces outils.

Sources : compose.yaml ; Caddyfile ; Makefile ; webpack.config.js ; phpunit.dist.xml.

08 Architecture de l'application

L'application suit une séparation de responsabilités proche du modèle MVC. Les entités représentent les données métier, les repositories centralisent les recherches, les contrôleurs coordonnent une requête et Twig construit la réponse HTML. Les formulaires participent à la transformation des valeurs reçues et à leur validation.

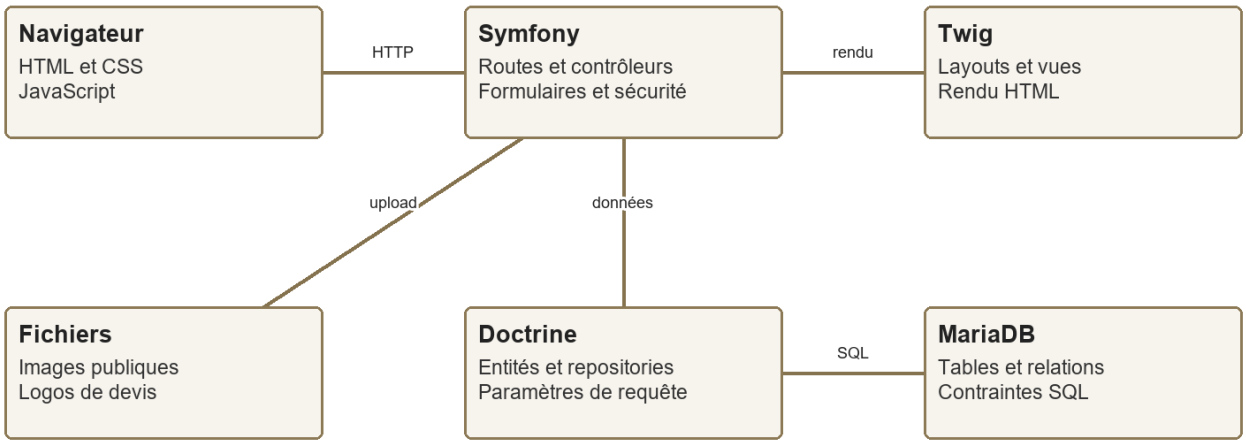


Figure 1 Architecture reconstituée à partir du code et de la configuration

Une visite de la galerie illustre ce fonctionnement. Le routeur sélectionne ProductController::index. Symfony lui fournit les repositories nécessaires. Le contrôleur demande les produits actifs, récupère les catégories et transmet ces deux collections au template. Le navigateur reçoit le HTML produit et charge les fichiers CSS et JavaScript compilés.

L'injection de dépendances permet de demander EntityManagerInterface, SluggerInterface ou MailerInterface sans construire leurs implémentations dans chaque contrôleur. services.yaml active l'autowiring et l'autoconfiguration pour les classes de src. Cette organisation réduit les couplages et rend possible l'extraction future d'un service métier testable.

La demande de devis concentre encore plusieurs responsabilités dans son contrôleur : composition du nom, stockage du logo, choix du statut, date et persistance. Le stockage du fichier est répété côté administration. Un service commun faciliterait la validation homogène, la gestion d'erreur et les essais.

Le schéma distingue MariaDB du stockage de fichiers. La base conserve un nom de fichier et non son contenu binaire. Cette séparation impose de sauvegarder les deux ensembles et de traiter les risques de fichiers orphelins lorsque l'écriture du fichier réussit mais que l'écriture SQL échoue.

Sources : src/Controller ; src/Repository ; src/Entity ; config/services.yaml.

09 Parcours et enchaînement des écrans

Le parcours principal commence par l'accueil, passe par les prestations ou la galerie et conduit à la demande de devis. Le menu donne également accès à la présentation de l'atelier et au contact. La connexion ouvre les fonctions associées au compte et, pour un administrateur, le back-office.

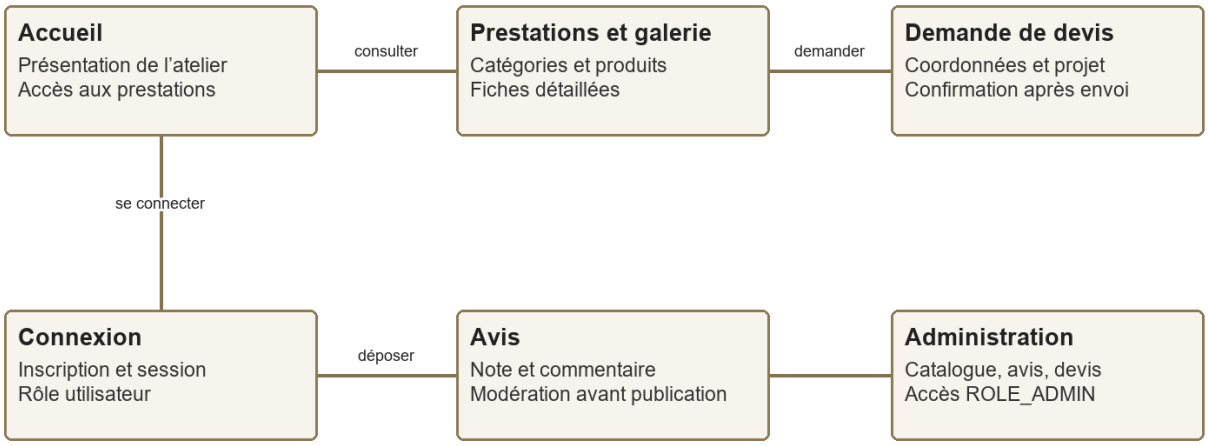


Figure 2 Enchaînement fonctionnel des écrans publics et privés

Les liens générés avec path utilisent des noms de routes. Un changement d'URL peut ainsi être effectué dans la définition de route sans réécrire tous les liens à la main. Le menu compare la route courante pour appliquer une classe active. Cette indication visuelle aide à situer l'utilisateur, mais son équivalent sémantique doit aussi être vérifié, par exemple avec aria-current.

Scénario représentatif

Une personne découvre une réalisation et souhaite une série de textiles personnalisés. Elle ouvre le formulaire, décrit le type d'article, l'origine du textile, le marquage et la quantité. Elle complète ses coordonnées, ajoute éventuellement un logo et transmet sa demande. Le serveur enregistre la demande, affiche une confirmation et redirige vers l'accueil. L'administrateur consulte ensuite la demande dans son interface.

Ce scénario ne comprend pas de paiement, de panier ni de devis tarifé automatiquement. La confirmation indique la réception de la demande ; elle ne vaut pas acceptation commerciale. Les termes employés dans le dossier respectent cette limite.

Les autres parcours doivent conserver des sorties compréhensibles : retour vers la galerie depuis une fiche, accès au formulaire de connexion et traitement d'une ressource absente. Les vérifications locales confirment une réponse 404 pour un slug produit inexistant et une redirection vers l'authentification pour l'administration anonyme.

Sources : header.html.twig ; contrôleurs Front et Admin ; contrôles HTTP du 11 octobre 2026.

10 Maquettage web et mobile

Le dossier du projet contient des captures graphiques, des documents de besoin client, des wireframes et des maquettes SVG pour ordinateur et mobile. Ces ressources servent à organiser les contenus et à comparer les intentions de présentation avec les écrans rendus par l'application.

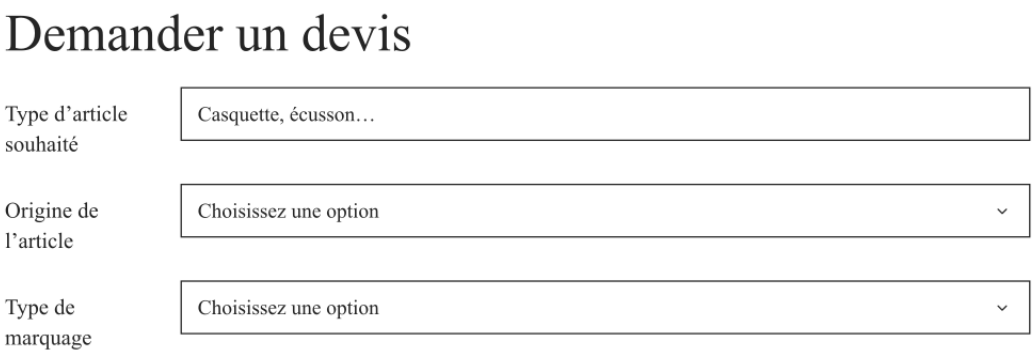


Figure 3 Extrait du wireframe web du devis fourni dans les supports SVG



Figure 4 Début de la maquette mobile du devis dont la version complète figure en annexe D

Le formulaire constitue un exemple d'adaptation : les champs doivent rester dans une seule colonne sur petit écran, avec des libellés lisibles et un bouton d'envoi accessible. La quantité, le type de marquage et les coordonnées sont séparés pour aider le visiteur à décrire son besoin.

Le document LEIA-ME des supports SVG précise que les variantes mobiles sont des adaptations et que l'importateur Figma n'a pas été exécuté dans Figma lors de leur préparation. Ces planches ne prouvent donc ni un prototype Figma validé, ni une recette mobile. Leur provenance doit rester explicite, notamment si elles ont été produites ou retravaillées avec assistance.

La prochaine validation consiste à comparer ces intentions avec le formulaire réellement utilisé, à vérifier le clavier et les messages d'erreur, puis à conserver la décision du commanditaire. Une maquette approuvée et une application conforme à cette maquette sont deux éléments complémentaires de la preuve.

Sources : output/design/nouveaux-svg ; LEIA-ME.md ; documents client dans output/design/Client.

11 Identité graphique et hiérarchie visuelle

L'identité visuelle met en avant l'artisanat textile. Le site associe un fond clair, des tons bruns et dorés et une image de machine à broder. Le logo et les photographies du projet constituent les ressources de marque disponibles. Leur droit d'utilisation doit être vérifié avant une diffusion publique définitive du dossier ou du site.



Figure 5 Accueil rendu depuis le noyau Symfony local à 1366 par 900 pixels

Le CSS définit notamment --bg-color à #fdfbf7, --primary-color à #b89753 et --text-dark à #2c2522. Le corps utilise Georgia avec des polices de repli à empattements. La navigation emploie une famille système sans empattement, en gras, pour distinguer les actions de navigation des textes de présentation.

Le bloc d'accueil place le titre, l'accroche et le bouton de découverte des prestations avant le texte de présentation. Cette hiérarchie vise à rendre la proposition de l'atelier compréhensible dès le premier écran. Les catégories et les réalisations complètent ensuite le parcours d'exploration.

Une palette cohérente ne garantit pas des contrastes suffisants. Le doré associé à du texte blanc doit être mesuré dans chaque composant et dans ses états survolé, actif et désactivé. Le thème sombre ajoute d'autres couples de couleurs à contrôler. Aucun taux de conformité RGAA n'est annoncé sur la seule base du choix des couleurs ou de Bootstrap.

Sources : assets/styles/page/page.css ; assets/images ; front/home/index.html.twig ; capture locale.

12 Réalisation des interfaces statiques

Twig fournit une structure commune aux pages publiques. Le layout contient l'en-tête, les messages flash, le bloc de contenu et le pied de page. Les pages filles remplacent les blocs title et body. Cette réutilisation réduit les divergences de navigation et facilite une correction commune, par exemple l'ajout de la langue ou du viewport.

Extrait du dépôt • templates/front/product/index.html.twig • lignes 26 à 30

```
26  <div class="row row-cols-1 row-cols-md-2 row-cols-lg-3 g-4">
27      {% for product in products %}
28          <div class="col">
29              <div class="card h-100 shadow-sm border-0 rounded-4 overflow-hidden
    product-card">
30                  {% if product.image is defined and product.image %}
```

Cet extrait associe la boucle de produits à la grille Bootstrap. Une colonne est prévue par défaut, deux à partir du point de rupture md et trois à partir de lg. Le HTML final présente chaque réalisation dans une carte avec image, titre, description courte et prix indicatif. Le titre du produit devient le texte alternatif de son image lorsqu'une image est présente.

L'intégration est dite statique du point de vue de la structure HTML et CSS, même si Twig reçoit des données serveur. La partie dynamique côté navigateur correspond à un autre niveau : événements, modification du DOM et composants JavaScript. Cette distinction évite de réduire CP4 à une simple boucle exécutée avant l'envoi de la page.

Ressources et maintenance

Les liens et images utilisent les helpers de Symfony. Les fichiers transférés dans l'administration sont lus depuis uploads/products ou uploads/categories ; certaines données de démonstration peuvent également contenir une URL externe HTTPS. Le template prévoit un emplacement de remplacement lorsqu'aucune image n'est disponible.

La compilation Encore fournit les liens vers les fichiers générés. Le nom versionné d'une ressource en production permet de changer son URL quand son contenu change, ce qui aide à gérer le cache navigateur. Il faut cependant déployer les manifests et les fichiers correspondants ensemble.

Les 51 templates examinés passent le contrôle de syntaxe Twig. Ce résultat garantit la validité syntaxique des fichiers présents, mais ne détecte pas un nom de template demandé par un contrôleur alors que le fichier n'existe pas, ni une mauvaise donnée reçue à l'exécution.

Sources : front/base.html.twig ; front/product/index.html.twig ; webpack.config.js ; résultat lint:twig.

13 Adaptation aux petits écrans

Le projet utilise les classes de grille Bootstrap et un menu repliable. Ces mécanismes permettent de changer le nombre de colonnes et de regrouper les liens sur un écran étroit. Les maquettes mobiles conservées montrent l'intention d'une lecture verticale et de formulaires accessibles sans zoom.

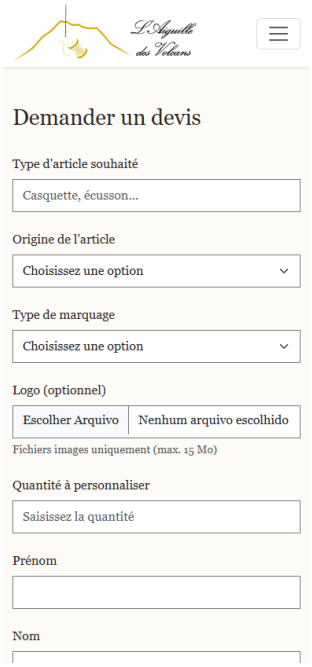


Figure 6 Formulaire rendu dans une fenêtre de 390 par 844 pixels sans modifier le HTML source

Cette capture utilise un navigateur de bureau redimensionné. Elle confirme le rendu dans cette fenêtre ; elle ne remplace pas une émulation de viewport mobile ou un essai sur un téléphone réel. Le layout public ne contient pas de balise meta viewport et la balise html ne déclare pas lang="fr". L'absence de viewport peut empêcher le navigateur mobile d'utiliser la largeur attendue pour ses points de rupture.

La correction prévue consiste à déclarer width=device-width et initial-scale=1 dans le layout, puis à vérifier les pages à plusieurs largeurs. Le contrôle doit porter sur les champs, les boutons, le débordement horizontal, les images et la navigation. Les tableaux de gestion demandent un traitement spécifique, car leur largeur peut dépasser celle d'un petit écran.

La taille d'une capture et la présence d'une grille responsive ne suffisent donc pas à déclarer l'application entièrement adaptée au mobile. Les essais clavier, le zoom, l'orientation et les messages d'erreur restent à intégrer à une recette traçable.

Sources : front/base.html.twig ; header.html.twig ; capture locale redimensionnée ; maquettes mobiles.

14 Interactions JavaScript

Trois modules personnalisés complètent les composants de Bootstrap : theme.js, image-preview.js et lightbox.js. app.js importe ces modules et les ressources JavaScript de Bootstrap. Chaque module attend DOMContentLoaded et recherche les éléments nécessaires avant d'installer ses écouteurs.

Extrait du dépôt • assets/scripts/theme.js • lignes 25 à 30

```
25  const setTheme = (theme) => {
26      const isDark = theme === darkTheme;
27      document.body.classList.toggle('dark-mode', isDark);
28      updateThemeButton(isDark);
29  };
30
```

Le thème agit sur la classe dark-mode du body et sur les variables CSS associées. Le choix est conservé dans localStorage sous la clé site-theme. Le module actualise aussi les logos, l'icône et le libellé aria-label du bouton. Ces mises à jour rendent l'état compréhensible au-delà de la seule couleur de l'icône.

L'aperçu d'image écoute l'événement change du champ portant data-image-input. Il vérifie les types d'éléments, récupère le premier fichier et crée une URL temporaire avec URL.createObjectURL. Avant un nouvel aperçu, l'ancienne URL est libérée par URL.revokeObjectURL. L'image reste locale au navigateur jusqu'à l'envoi du formulaire.

La lightbox ouvre une modale Bootstrap à partir d'un élément portant data-lightbox-src. Elle copie le chemin de l'image et son titre dans la modale, puis vide ces valeurs à la fermeture. L'emploi de textContent pour le titre évite d'interpréter cette chaîne comme du balisage HTML.

Limites à tester

Un aperçu réussi ne prouve pas qu'un fichier est accepté par le serveur. Les contrôles de taille et de type MIME doivent rester côté formulaire Symfony. Le stockage local peut être indisponible dans certains contextes de navigateur ; une gestion d'erreur autour de localStorage rendrait le changement de thème plus robuste.

Les parcours de modale doivent être testés au clavier, notamment l'ouverture, la touche Échap, le focus à la fermeture et le nom accessible du bouton. Les modules sont présents et analysés ; une batterie automatisée d'essais navigateur n'est pas encore livrée avec le projet.

Sources : assets/scripts/app.js ; theme.js ; image-preview.js ; lightbox.js ; front/base.html.twig.

15 Catalogue et navigation par catégorie

La galerie générale récupère les produits actifs avec un tri décroissant par date de création. La fiche utilise le slug et l'état actif pour trouver le produit. Si aucune entité ne correspond, le contrôleur crée une réponse 404. Les catégories forment un second moyen de parcourir les réalisations.

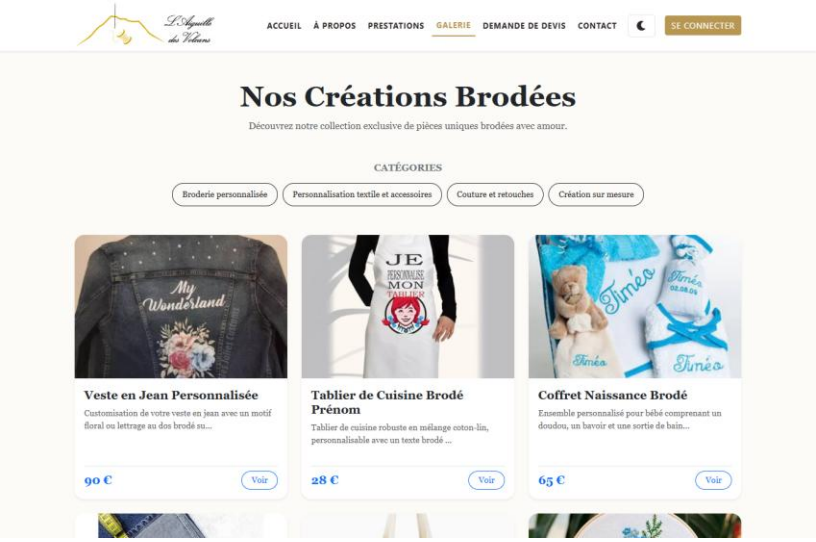


Figure 7 Galerie générale rendue localement à partir des données disponibles

Deux chemins de catégorie coexistent. Le premier, /category/{slug}, charge la catégorie et rend front/category/show.html.twig. Il est utilisé par les liens de catégorie de la galerie. Le second, /product/category/{slug}, appelle findByCategorySlug mais demande front/product/category.html.twig, absent du dépôt examiné. L'essai local sur une catégorie existante obtient une erreur 500 sur ce second chemin.

La première vue parcourt category.products sans vérifier isActive. Il faut donc harmoniser la récupération des produits publics et conserver une seule règle de filtrage. Une correction peut réutiliser la requête du repository dans le contrôleur de catégorie et transmettre explicitement les produits visibles au template.

La méthode searchByKeyword est disponible dans le repository, mais aucun champ de recherche relié à cette méthode n'a été démontré dans le parcours analysé. La recherche figure dans les possibilités d'évolution, pas dans les fonctionnalités validées.

Enfin, findBy et findAll chargent actuellement des listes sans pagination dans plusieurs écrans. Pour un petit catalogue, cela simplifie le développement. À mesure que les contenus augmentent, une pagination et le contrôle des requêtes deviennent nécessaires pour limiter la taille des réponses et le coût de lecture des relations.

Sources : ProductController ; CategoryController ; ProductRepository ; category/show.html.twig ; essais HTTP.

16 Modélisation conceptuelle des données

Le modèle distingue cinq objets métier : utilisateur, produit, catégorie, avis et demande de devis. Cette séparation correspond aux fonctions administratives et évite de mélanger le contenu du catalogue avec les échanges commerciaux ou les informations d'authentification.

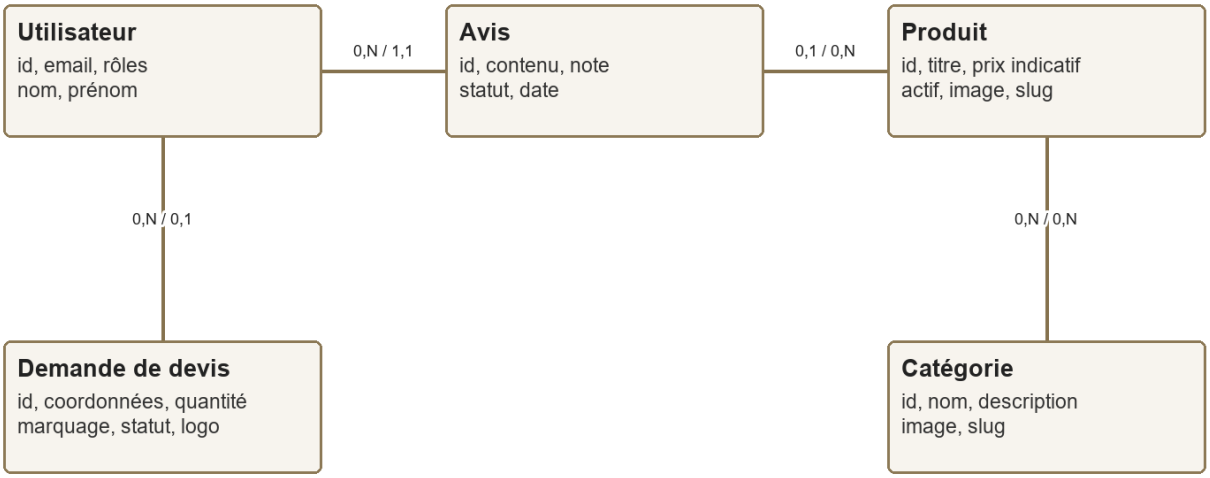


Figure 8 Modèle conceptuel reconstitué selon les associations et leur optionalité dans les entités

Les cardinalités de la figure se lisent dans l'ordre des deux objets reliés. Un utilisateur peut avoir zéro à plusieurs avis, tandis qu'un avis appartient à un utilisateur obligatoire. Un avis peut concerner zéro ou un produit et un produit peut recevoir plusieurs avis. L'absence de produit permet un témoignage général sur l'atelier.

Un utilisateur peut être associé à plusieurs demandes de devis, mais une demande peut être anonyme au sens de l'absence de compte lié. Elle possède néanmoins des coordonnées nominatives. Le fait de permettre user_id NULL ne rend donc pas la demande dépourvue de données personnelles.

La relation produit catégorie est plusieurs à plusieurs. Un produit peut appartenir à plusieurs catégories et une catégorie regroupe plusieurs produits. Cette association demande une table de liaison dans le modèle relationnel. Les collections et méthodes add/remove entretiennent les deux côtés de la relation en mémoire.

Le diagramme est une synthèse documentaire du modèle actuel, et non la preuve d'un MCD initial validé à une date antérieure. Les premiers schémas et exports SQL sont conservés dans output/docs, mais les entités et migrations actuelles servent de référence pour les détails présentés. Cette distinction évite de montrer un ancien schéma comme s'il décrivait la base vérifiée.

Sources : src/Entity/User.php ; Product.php ; Category.php ; Review.php ; QuoteRequest.php.

17 Modèle physique et dictionnaire des données

Le modèle physique utilise les tables user, product, category, product_category, review et quote_request. Une table technique messenger_messages apparaît aussi dans les migrations ; elle n'est pas un objet métier du catalogue. Les clés primaires identifient les enregistrements, les clés étrangères maintiennent les relations et un index unique protège l'adresse électronique des comptes.

Table	Colonnes significatives	Contraintes et interprétation
user	id INT ; email VARCHAR(180) ; roles JSON ; password VARCHAR(255) ; first_name ; last_name ; created_at ; is_verified	Email unique ; mot de passe haché
product	id ; title ; description LONGTEXT ; indicative_price INT NULL ; image ; slug ; is_active ; created_at	Prix facultatif ; état de visibilité
category	id ; name ; image NULL ; slug ; description NULL	Classement des réalisations
product_category	product_id ; category_id	Clé primaire composée ; deux clés étrangères avec cascade
review	id ; content ; status ; rating ; created_at ; product_id NULL ; user_id	Utilisateur obligatoire ; produit facultatif
quote_request	id ; name ; email ; message ; status ; created_at ; user_id NULL ; détails textile et logo	Coordonnées autonomes ; compte facultatif

Dans quote_request, les détails ajoutés comprennent article_type, article_origin, marking_type, logo, quantity, organization_type, organization_name et phone. Le téléphone dispose d'une longueur SQL de 50 caractères. Les champs optionnels permettent de conserver des demandes antérieures à l'enrichissement du formulaire.

Les slugs ne possèdent pas de contrainte d'unicité dans les entités examinées. Deux titres identiques peuvent donc produire un slug ambigu. Un contrôle applicatif et un index unique sont à envisager ensemble. La convention du prix mérite également d'être documentée : le template affiche directement l'entier suivi du symbole euro ; le code observé ne réalise pas de conversion depuis des centimes.

La suppression d'un produit peut supprimer ses avis associés par cascade. La suppression d'un utilisateur lié à des avis ou à des devis requiert une politique explicite, car toutes les relations ne prévoient pas la même action SQL. Une contrainte relationnelle protège l'intégrité, mais ne décide pas à elle seule de la politique de conservation.

Sources : attributs ORM ; migrations ; doctrine:schema:validate exécuté le 11 octobre 2026.

18 Migrations et données de démonstration

Huit fichiers de migration retracent la création des tables, les relations, le champ de vérification de compte, l'optionalité du produit dans un avis, l'enrichissement des devis et la normalisation de leurs statuts. Une migration décrit une transformation versionnée du schéma ou des données ; elle doit être examinée avant son application sur une base existante.

Extrait du dépôt • migrations/Version20260919182000.php • lignes 19 à 28

```
19      $this->addSql("
20          UPDATE quote_request
21          SET status = CASE
22              WHEN status IN ('processing') THEN 'processing'
23              WHEN status IN ('finished', 'processed', 'approved') THEN 'finished'
24              ELSE 'new'
25          END
26      ");
27  }
28
```

La normalisation remplace différentes anciennes valeurs par new, processing ou finished. Son commentaire de retour arrière précise que les anciennes valeurs ne peuvent pas être restaurées de façon fiable. Cela justifie une sauvegarde avant exécution et un plan de restauration complet, plutôt que l'idée qu'un down annule toujours sans perte toute migration.

Une migration précédente remplace category_product par product_category en supprimant l'ancienne table de liaison. Si elle contient déjà des associations, une migration de données est nécessaire pour les conserver. Le schéma final validé ne prouve pas que toutes les transformations historiques sont sans perte sur n'importe quelle base.

Jeux de données

AppFixtures définit quatre catégories métier, un administrateur de démonstration, 50 utilisateurs, 30 produits, 40 avis et 15 demandes de devis. Foundry et Faker produisent les valeurs et les associations. Ces quantités décrivent le générateur ; elles ne sont pas un relevé de la base active.

Les mots de passe de démonstration sont hachés par UserFactory. Leur présence sous une forme connue dans les fixtures implique que ces comptes ne doivent pas être chargés tels quels en production. Les images aléatoires externes utilisées par certaines factories sont pratiques pour tester une mise en page, mais elles ne représentent pas les réalisations réelles de l'atelier.

Les données d'essai doivent rester reconnaissables, reproductibles et séparées des demandes commerciales. Aucune recharge des fixtures ni réinitialisation de la base n'a été effectuée pour préparer ce dossier. La validation exécutée a confirmé que le schéma courant correspond au mappage Doctrine.

Sources : migrations ; AppFixtures ; factories Foundry ; contrôle Doctrine du 11 octobre 2026.

19 Accès aux données avec Doctrine

Les repositories étendent ServiceEntityRepository et utilisent ManagerRegistry. Ils regroupent les recherches liées à une entité. Le contrôleur peut ainsi exprimer une intention métier, par exemple obtenir les dernières réalisations actives, sans assembler lui-même une requête SQL dans la méthode d'affichage.

Extrait du dépôt • src/Repository/ProductRepository.php • lignes 29 à 40

```
29  public function findByCategorySlug(string $slug): array
30  {
31      return $this->createQueryBuilder('p')
32          ->innerJoin('p.categories', 'c')
33          ->andWhere('p.isActive = :active')
34          ->andWhere('c.slug = :slug')
35          ->setParameter('active', true)
36          ->setParameter('slug', $slug)
37          ->orderBy('p.createdAt', 'DESC')
38          ->getQuery()
39          ->getResult();
40  }
```

La jointure relie les produits à leurs catégories. Les valeurs active et slug sont transmises comme paramètres et ne sont pas concaténées dans la structure de la requête. Le tri décroissant sur createdAt présente d'abord les dernières créations. Cette méthode correspond à une lecture relationnelle SQL exécutée par Doctrine.

findActiveRecent limite le nombre de résultats, huit à l'appel depuis l'accueil. ReviewRepository::findLatestApproved utilise le statut approuvé, un tri par date et une limite de six. Ces méthodes rendent explicites des règles qui seraient moins visibles dans un simple findAll.

L'EntityManager gère la persistance. persist indique qu'une nouvelle entité doit être suivie ; flush synchronise les changements vers la base. Pour une entité déjà chargée et suivie, le contrôleur peut modifier ses propriétés puis appeler flush. remove planifie une suppression, qui reste soumise aux contraintes SQL et aux protections de la route.

Limites et évolutions

La paramétrisation réduit le risque d'injection pour ces requêtes, mais elle ne remplace pas les contrôles de droits et de validation. Une requête correctement construite peut encore exposer un enregistrement que l'utilisateur ne devrait pas voir. L'état actif du produit et le statut de l'avis restent donc des conditions métier à appliquer systématiquement.

Les listes administratives ne sont pas paginées. Des chargements de relations dans des boucles peuvent aussi augmenter le nombre de requêtes ; leur impact doit être mesuré avec le profiler et un volume représentatif. Aucun résultat de mesure SQL ou de performance n'est inventé dans ce dossier.

Sources : ProductRepository ; ReviewRepository ; contrôleurs de création, modification et suppression.

20 Conception du formulaire de devis

La demande de devis constitue la fonctionnalité centrale du projet. Elle collecte les renseignements nécessaires à un premier échange sans calculer un prix final. QuoteRequestType décrit les champs et les associe à QuoteRequest, à l'exception des champs explicitement non mappés.

Groupe	Champs	Traitement
Besoin textile	articleType, articleOrigin, markingType, quantity	Choix encadrés et quantité minimale
Coordonnées	firstName, lastName, email, phone	Nom composé dans le contrôleur
Organisation	organizationType, organizationName	Typologie et nom facultatif
Projet	message, logoFile	Texte libre et pièce jointe facultative
Données serveur	status, createdAt, user	Définies par le traitement

firstName et lastName sont non mappés parce que l'entité conserve un seul champ name. logoFile est également non mappé : un UploadedFile n'est pas directement stocké dans la colonne logo, qui conserve seulement le nom attribué au fichier. Cette distinction sépare les données temporaires de formulaire des valeurs persistantes.

La quantité reçoit GreaterThanOrEqual(1). Les ChoiceType limitent notamment le marquage à broderie, flocage ou sublimation et l'origine du textile à client ou atelier. Le fichier accepte plusieurs types d'image avec une limite applicative de 15 Mo. Ces contraintes sont évaluées pendant la validation du formulaire.

Le type EmailType et l'attribut required améliorent le contrôle dans le navigateur. Les essais de préparation du dossier montrent cependant que l'e-mail invalide et le prénom vide passent la validation Symfony lorsque le formulaire est soumis directement en mémoire. Il manque des contraintes serveur telles que Email et NotBlank aux endroits concernés.

La correction doit porter sur les contraintes de validation, puis être vérifiée avec les mêmes données. Une limite de longueur explicite doit aussi être alignée sur les colonnes SQL. Les types HTML, les attributs d'interface et la nullabilité de la base interviennent à des niveaux différents et ne doivent pas être confondus.

Sources : QuoteRequestType ; QuoteRequest ; essais en mémoire détaillés au chapitre 29.

21 Traitement serveur de la demande

QuoteRequestController::new crée l'entité, associe éventuellement l'utilisateur courant, construit le formulaire et traite la requête. Le bloc principal ne s'exécute que lorsque le formulaire est soumis et valide. Le contrôleur compose le nom à partir des deux champs non mappés, traite le logo, fixe la date et choisit le statut initial.

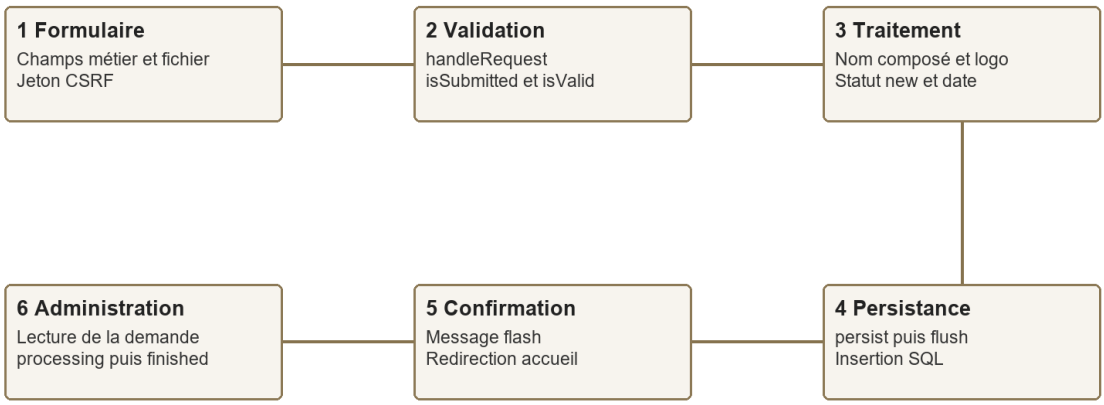


Figure 9 Séquence logique du devis du formulaire au traitement administratif

Extrait du dépôt • src/Controller/Front/QuoteRequestController.php • lignes 54 à 62

```
54  $quoteRequest->setCreatedAt(new \DateTimeImmutable());
55  $quoteRequest->setStatus(QuoteRequest::STATUS_NEW);
56
57  $entityManager->persist($quoteRequest);
58  $entityManager->flush();
59
60  $this->addFlash('success', 'Votre demande de devis a été envoyée avec succès !');
61
62  return $this->redirectToRoute('app_home');
```

L'appel à persist puis flush enregistre la demande. Le message flash est suivi d'une redirection vers l'accueil. Cette séquence évite qu'un simple rafraîchissement de la page de confirmation renvoie directement le même POST, même si elle ne fournit pas une garantie complète d'idempotence en cas de double soumission.

Pour le logo, un nom aléatoire de 16 octets convertis en hexadécimal est associé à une extension déterminée à partir du fichier. Le contrôleur crée le dossier si nécessaire et déplace le fichier. Le nom aléatoire ne constitue pas un mécanisme de confidentialité : le dossier de destination se trouve dans public.

Le préremplissage appelle setName sur l'entité lorsque l'utilisateur est connecté, mais les champs firstName et lastName sont non mappés et ne reçoivent pas de données explicites. Leur préremplissage n'est donc pas assuré par ce seul appel. Un essai connecté doit confirmer le comportement avant de l'annoncer comme acquis.

Enfin, le fichier est déplacé avant flush. Un échec SQL pourrait laisser un fichier sans demande associée. Un service de stockage avec traitement compensatoire et journalisation permettrait de mieux gérer ce cas.

Sources : QuoteRequestController::new ; config/services.yaml ; QuoteRequestType.

22 Suivi administratif des devis

L'administration propose une liste des demandes, une fiche détaillée, un formulaire de modification, une création manuelle et une suppression protégée par jeton CSRF. Le tableau de bord affiche également le nombre de nouvelles demandes et les cinq demandes les plus récentes.

Le suivi repose sur new, processing et finished. L'administrateur choisit une valeur dans un ChoiceType, et l'entité refuse un statut extérieur à la liste autorisée. Ce mécanisme empêche une valeur inattendue, mais ne vérifie pas qu'un devis terminé possède une réponse envoyée au client : aucune réponse commerciale structurée n'est stockée dans ce modèle.

Conséquences pour l'usage métier

Le statut permet de repérer les demandes à examiner. Il n'est pas une preuve d'envoi de devis, de signature, de paiement ou de livraison. Pour une activité plus importante, une date de traitement, un historique des changements et une référence de réponse pourraient compléter le suivi. Ces ajouts nécessiteraient des règles claires sur les droits et la conservation.

Le formulaire administratif expose davantage de champs que le formulaire public, notamment l'utilisateur lié et la date de création. Cette souplesse peut aider à reprendre une demande reçue autrement, mais elle autorise aussi la modification d'informations de traçabilité. Il serait préférable de distinguer la date initiale de réception d'une date de dernière modification.

Le stockage de logo répète une partie du traitement public. Une centralisation éviterait de faire évoluer deux implémentations séparées pour la sécurité des fichiers. La suppression actuelle de l'entité n'est pas accompagnée d'un mécanisme visible de suppression de la pièce jointe ; le cycle de vie des fichiers doit donc être défini.

Les coordonnées présentes dans les demandes sont des données personnelles. Pour cette raison, le dossier n'utilise pas de capture récente des listes privées contenant des données réelles. La preuve technique repose sur le contrôleur, les formulaires et les règles d'accès, tandis que la démonstration pourra utiliser des demandes fictives préparées pour l'examen.

Un critère d'acceptation concret serait le suivant : après l'enregistrement d'une demande fictive, l'administrateur la retrouve dans la liste, consulte toutes les informations et peut enregistrer processing. La modification doit persister après rechargement sans changer involontairement l'identité du demandeur. Ce parcours complet reste à consigner dans une recette dédiée.

Sources : AdminQuoteRequestController ; AdminQuoteRequestType ; DashboardController ; templates admin_quote_request.

23 Avis et modération

Un avis comprend un contenu, une note, un statut, une date, un utilisateur obligatoire et un produit facultatif. ReviewController exige une authentification complète avant la création. Il choisit lui-même l'utilisateur, le statut pending et la date, ce qui empêche le formulaire public d'attribuer directement l'avis à une autre personne ou de le publier sans modération.

Extrait du dépôt • src/Controller/Front/ReviewController.php • lignes 40 à 48

```
40  $review->setUser($this->getUser());
41
42  $review->setStatus(Review::STATUS_PENDING);
43  $review->setCreatedAt(new \DateTimeImmutable());
44
45  $entityManager->persist($review);
46  $entityManager->flush();
47
48  $this->addFlash('success', 'Merci ! Votre avis a été soumis et sera publié après
    validation.');
```

Le formulaire borne la note entre 1 et 5 avec Range. Il propose un produit facultatif pour permettre un témoignage général. La liste publique sélectionne uniquement les avis approuvés et l'accueil limite leur nombre à six. Le statut est une règle de publication, indépendamment de la validité technique des autres champs.

L'administration peut approuver, rejeter, modifier ou supprimer un avis. Le setter de Review vérifie la liste pending, approved et rejected. Comme pour les devis, il s'agit d'un ensemble de valeurs autorisées et non d'un historique de modération. La date, l'auteur et le motif d'une décision ne sont pas enregistrés dans une structure dédiée.

Points de vigilance

L'identification par compte ne prouve pas qu'un avis provient d'un achat ou d'une prestation réellement réalisée. Le site n'enregistre pas de commande et ne doit pas employer une mention de type « achat vérifié » sans mécanisme correspondant. La vérification d'adresse électronique est également incomplète dans le parcours actuel.

Le choix de produit du formulaire utilise EntityType sans filtrage visible sur isActive. Il peut donc proposer des produits désactivés selon les données présentes. L'ajout d'un query_builder au champ permettrait d'aligner ce choix avec la règle du catalogue public.

Le contenu textuel doit recevoir des contraintes explicites de présence et de longueur. L'échappement Twig protège l'affichage HTML dans son contexte habituel, mais ne remplace ni la modération ni la protection contre les soumissions abusives. Des tests doivent vérifier le dépôt anonyme, la borne de note, l'absence de publication immédiate et l'affichage après décision administrative.

Sources : Review ; ReviewType ; ReviewController ; ReviewRepository ; AdminReviewController.

24 Authentification et autorisations

L'utilisateur est identifié par son adresse électronique. Le provider Doctrine recharge l'entité User et Symfony Security gère la connexion par formulaire. Le hachage des mots de passe utilise l'algorithme auto configuré pour PasswordAuthenticatedUserInterface. Le mot de passe saisi n'est pas conservé en clair dans la colonne password.

Extrait du dépôt • config/packages/security.yaml • lignes 40 à 44

```
40  access_control:
41      - { route: '^admin_', roles: ROLE_ADMIN }
42      # - { path: ^/admin, roles: ROLE_ADMIN }
43      - { path: ^/, roles: PUBLIC_ACCESS }
44      # - { path: ^/profile, roles: ROLE_USER }
```

L'ordre des règles d'accès est déterminant : la première règle correspondante est appliquée. Ici, les noms de routes commençant par admin_ exigent ROLE_ADMIN, avant la règle publique générale. DashboardController ajoute aussi un attribut IsGranted. Les pages de dépôt d'avis appellent denyAccessUnlessGranted pour exiger un utilisateur authentifié.

Le formulaire de connexion active le CSRF. Le fournisseur vérifie l'adresse de connexion et le mot de passe. La vue affiche les erreurs d'authentification et permet de retrouver l'adresse précédemment saisie. Les droits de l'interface ne sont jamais déduits de la seule présence d'un bouton visible.

Inscription et vérification du compte

RegistrationController hache le plainPassword, persiste le compte, puis connecte l'utilisateur. User possède un indicateur isVerified et un service EmailVerifier capable de générer et de contrôler un lien signé. Toutefois, l'envoi du courriel de confirmation est commenté dans le contrôleur. La validation de l'adresse électronique ne peut donc pas être présentée comme active de bout en bout.

La contrainte de longueur minimale du mot de passe est actuellement de six caractères. Une politique plus robuste et une limitation des tentatives de connexion sont à envisager. Aucune configuration de login_throttling n'a été relevée dans le fichier de sécurité examiné.

La méthode __serialize transforme le hachage de mot de passe pour son stockage en session. Cette transformation ne remplace pas le hachage de sécurité du mot de passe en base. La distinction est importante pour expliquer au jury le rôle de chaque mécanisme et éviter d'attribuer à un contrôle une protection qu'il ne fournit pas.

Sources : User ; RegistrationController ; RegistrationFormType ; EmailVerifier ; security.yaml.

25 Administration du catalogue et des comptes

Les contrôleurs administratifs séparent les opérations de liste, création, consultation, modification et suppression. Symfony résout les entités à partir des identifiants de route et les formulaires portent les champs modifiables. Les écritures n'interviennent qu'après soumission et validation du formulaire.

Pour un produit, le contrôleur fixe la date à la création et génère le slug à partir du titre avec SluggerInterface. Le fichier image est non mappé dans le formulaire, puis son nom est enregistré dans l'entité après déplacement. Le même principe est employé pour les catégories. Les images acceptées sont JPEG, PNG ou WebP et la taille maximale déclarée est de 2 Mo.

Extrait du dépôt • src/Controller/Admin/AdminProductController.php • lignes 113 à 121

```
113  public function delete(Request $request, Product $product, EntityManagerInterface
     $entityManager): Response
114  {
115      if ($this->isCsrfTokenValid('delete'.$product->getId(),
     $request->getPayload()->getString('_token'))) {
116          $entityManager->remove($product);
117          $entityManager->flush();
118          $this->addFlash('success', 'Produit supprimé avec succès.');
119      }
120
121      return $this->redirectToRoute('admin_product_index', [],
     Response::HTTP_SEE_OTHER);
```

La suppression utilise POST et un jeton associé à l'identifiant. Le contrôle du jeton protège l'action contre une requête forgée, tandis que la règle d'accès à la route vérifie l'autorisation administrative. Les deux protections sont complémentaires.

Limites relevées

La génération de slug n'est pas accompagnée d'une unicité garantie. Le remplacement d'image ne supprime pas explicitement l'ancien fichier. Dans le contrôleur de catégorie, une exception de déplacement est interceptée sans action dans le catch, puis le nom peut tout de même être affecté à l'entité. Cela peut produire une référence à une image absente et une confirmation trompeuse.

AdminUserType lie directement le champ password à l'entité. Le contrôleur essaie de conserver le hachage lorsqu'il n'est pas modifié et de hacher une nouvelle valeur. Un champ plainPassword non mappé, de type PasswordType et vide par défaut, serait plus clair et réduirait le risque d'afficher ou de manipuler le hachage dans le formulaire.

Les suppressions de comptes ou de produits doivent aussi être expliquées du point de vue métier. Une cascade SQL peut supprimer des avis, tandis qu'une autre relation peut bloquer l'opération. Le résultat attendu doit être fixé avant de présenter la suppression comme une opération sans conséquence sur les autres données.

Sources : contrôleurs Admin ; AdminProductType ; AdminCategoryType ; AdminUserType ; relations ORM.

26 Sécurité des données entrantes et des fichiers

La sécurité du projet repose sur plusieurs niveaux. L'authentification identifie un compte, les autorisations réservent l'administration à son rôle, les jetons CSRF protègent certaines écritures et les contraintes de formulaire contrôlent des valeurs. Doctrine paramètre les requêtes et Twig échappe les données dans les sorties HTML habituelles.

Risque	Protection observée	Limite et action attendue
Injection SQL	Paramètres Doctrine	Conserver ce principe pour les nouvelles requêtes
XSS dans les textes	Échappement Twig ; textContent	Vérifier les contextes et tout futur usage de raw
CSRF	Login, Forms et suppressions	Contact manuel sans contrôle visible
Accès administratif	Routes admin_ et dashboard	Vérifier chaque nouvelle route et chaque rôle
Fichier non souhaité	MIME, taille et nom généré	Logos SVG et stockage public à revoir
Données invalides	Choix et bornes numériques	Ajouter présence et validation d'e-mail serveur

Le formulaire de devis accepte image/svg+xml. Un SVG peut contenir du contenu actif ; vérifier son type MIME ne revient pas à assainir son contenu. Les logos sont stockés dans public/uploads/quote-logos. Le nom aléatoire réduit la prévisibilité mais n'impose aucun droit de lecture à une personne qui connaît l'URL. Le stockage privé avec téléchargement autorisé, ou une conversion vers un format raster sûr, est une amélioration prioritaire.

Le contact utilise directement les paramètres de requête et vérifie surtout qu'ils ne sont pas vides. L'adresse du visiteur devient l'expéditeur du message. Un expéditeur appartenant au domaine avec Reply-To pour l'utilisateur serait plus adapté à la délivrabilité. La validation serveur, le CSRF, une protection contre les abus et la journalisation des erreurs restent à renforcer.

Ces constats sont issus du code et des essais ciblés. Ils ne constituent pas un audit de pénétration et ne prouvent pas une exploitation en production. Aucune faille n'est déclarée corrigée lorsque le code examiné conserve encore le comportement décrit.

Sources : Forms, contrôleurs et services ; Symfony CSRF ; OWASP File Upload, consultés le 11 octobre 2026.

27 Accessibilité et données personnelles

L'accessibilité doit être vérifiée sur les parcours réels : navigation, consultation, formulaire, erreur et confirmation. Le code contient des éléments utiles, notamment des libellés de formulaire, des textes alternatifs sur les images du catalogue et des aria-label sur plusieurs commandes. Ils constituent des points d'appui, pas une preuve globale de conformité.

Le layout public ne déclare pas la langue française. L'absence de viewport est également relevée. Les contrastes du doré, la visibilité du focus, l'ordre des titres, l'usage au clavier de la modale et l'association des erreurs aux champs doivent être testés. Les correctifs devraient être vérifiés dans les thèmes clair et sombre, puis avec zoom et lecteur d'écran sur les parcours essentiels.

Le RGAA sert ici de cadre de contrôle technique. Aucun audit complet ni taux de conformité n'est fourni. La question des obligations réglementaires propres à l'atelier dépend de sa situation et ne peut pas être déduite du seul dépôt de code.

Données collectées

Le site traite des noms, adresses électroniques, téléphones, messages, fichiers joints et données de compte. Une demande sans compte contient toujours des données personnelles. Le besoin de devis justifie d'examiner chaque champ et de limiter la collecte aux informations utiles à la réponse.

Une information au moment de la collecte doit préciser le responsable, la finalité, la base légale, les destinataires, la durée de conservation et les modalités d'exercice des droits. Le consentement n'est pas à utiliser automatiquement comme justification de tous les traitements. La base légale et les durées doivent être arrêtées avec la responsable de l'activité.

Le dépôt ne suffit pas à établir une procédure effective d'accès, de rectification ou de suppression des données. Il faut aussi définir le traitement des sauvegardes et des logos devenus inutiles. Les coordonnées ne doivent pas apparaître dans des captures publiques ou des journaux partagés sans nécessité.

Le thème utilise localStorage. Aucun outil d'analyse publicitaire n'est démontré par cette seule fonction. L'inventaire réel des cookies et stockages doit être vérifié sur le site déployé avant de déterminer les informations ou mécanismes de consentement appropriés.

Sources : templates ; theme.js ; entités ; RGAA et principes CNIL ; absence d'audit formel dans les livrables.

28 Stratégie de vérification

La préparation du dossier a distingué trois niveaux de preuve : inspection des sources, contrôles exécutés et essais proposés. Cette distinction évite de transformer une possibilité du framework en résultat obtenu. Les contrôles exécutés datent du 11 octobre 2026 et concernent le dépôt local, pas le serveur public.

La première étape vérifie la syntaxe des vues et des configurations. La deuxième compare le mappage ORM et le schéma de la base. La troisième exerce plusieurs routes par le noyau Symfony. La quatrième soumet le formulaire de devis en mémoire avec des entrées valides et invalides.

Conditions des essais

Les requêtes locales utilisent BrowserKit avec une désactivation de la maintenance uniquement dans le processus de vérification. Les fichiers de configuration du site n'ont pas été modifiés. Les captures rendent les réponses HTML avec les ressources locales ; les accès externes non nécessaires ont été bloqués pendant ce rendu. Les images dépendant de services externes peuvent donc manquer dans cette méthode de capture.

Les essais de formulaire utilisent le véritable QuoteRequestType dans le conteneur de test, avec CSRF désactivé uniquement pour ces soumissions en mémoire. Ils vérifient la validation des données et non le fonctionnement complet de la protection CSRF. Aucune demande n'a été enregistrée en base, aucun logo n'a été transféré et aucun courriel n'a été envoyé par ces essais.

Recette à compléter

Une recette complète doit couvrir un devis envoyé par le navigateur, sa présence en base, son affichage administratif et le changement de statut. Elle doit aussi vérifier un fichier refusé, un jeton incorrect, le comportement connecté et la navigation mobile réelle. Pour les avis, les contrôles porteront sur le dépôt, l'attente de modération et la publication après validation.

Il reste nécessaire de tester un compte utilisateur sans droit administratif, puis un administrateur, avec des données fictives identifiables. Ces essais pourront devenir des tests fonctionnels conservés dans tests. La suite actuelle n'en contient pas : PHPUnit répond « No tests executed! ». Ce résultat n'est ni un échec de toutes les fonctionnalités ni une validation de leur bon fonctionnement.

Un compte rendu de recette doit conserver la version du code, l'environnement, les données utilisées, le résultat et les pièces justificatives. Après correction, les cas qui ont échoué doivent être rejoués avant d'élargir la campagne. Cette pratique transforme un incident observé en protection durable contre sa réapparition.

Sources : contrôles exécutés ; scripts dans tmp/dossier ; configuration PHPUnit et Symfony.

29 Jeux d’essai et résultats obtenus

Le jeu de référence utilise un besoin fictif : 12 polos fournis par le client, marquage broderie, nom Camille Exemple, organisme de type association, adresse camille@example.test et description d’une série de douze polos. Aucun fichier n’est joint. Chaque variante change un paramètre pour isoler le contrôle étudié.

Essai	Résultat attendu	Résultat obtenu
Données de référence	Formulaire valide	Valide
Quantité 0	Refus de la quantité	Refus avec GreaterThanOrEqual
Marquage inconnu	Refus du choix	Refus par ChoiceType
E-mail « invalide »	Refus de l’adresse	Accepté ; écart confirmé
Prénom vide	Refus du champ	Accepté ; écart confirmé
Statuts new, processing, finished	Acceptés par le setter	Acceptés
Statut approved sur un devis	Refus par le setter	InvalidArgumentException

Les deux écarts de validation montrent que les restrictions HTML du navigateur ne constituent pas un contrôle suffisant côté serveur. La correction prévue est l’ajout de contraintes explicites, suivi de la répétition du même jeu d’essai. Les résultats ci-dessus sont obtenus avant correction et ne doivent pas être présentés comme résolus.

Contrôle exécuté	Résultat	Portée
Syntaxe Twig	51 fichiers valides	Fichiers présents
Syntaxe YAML	26 fichiers valides	Configuration analysable
Schéma Doctrine	Mappage et base synchronisés	Structure courante
PHPUnit	Aucun test exécuté	Pas de suite livrée
GET accueil, galerie, devis, login	200	Réponses locales
GET admin et dépôt d’avis anonyme	302	Authentification demandée
GET produit inexistant	404	Ressource absente
GET product/category et slug existant	500	Template cible absent

Ces résultats ne mesurent ni les performances, ni la couverture du code, ni l’accessibilité. Ils donnent une base reproductible pour prioriser les correctifs. Le parcours d’écriture complet reste à exécuter sur une base dédiée aux essais.

Sources : http-results.json ; form-results.json ; sorties des contrôles du 11 octobre 2026.

30 Analyse des anomalies et traitement prévu

L'analyse a mis en évidence des écarts de natures différentes. Une erreur de route concerne la disponibilité d'un écran ; un manque de validation concerne la qualité des données ; le stockage public de logos concerne leur exposition. Les regrouper sous le terme « bugs » sans préciser leur impact rendrait les priorités moins lisibles.

Priorité	Constat	Correction et preuve attendue
Haute	E-mail invalide et prénom vide acceptés	Contraintes serveur ; cas refusés après correction
Haute	Logos publics et SVG autorisé	Stockage privé ou conversion ; essai d'accès
Haute	Route de catégorie en erreur 500	Template ou route harmonisée ; réponse 200
Haute	Produits inactifs dans une catégorie	Filtre commun ; produit caché partout
Moyenne	Langue et viewport absents	Layout corrigé ; essais mobiles et accessibles
Moyenne	Vérification e-mail inactive	Parcours complet ou retrait de la promesse
Moyenne	Erreurs de fichier et fichiers orphelins	Service commun ; essais d'échec
Moyenne	Aucune suite automatisée	Tests métier et fonctionnels dans le dépôt

Le diagnostic de la catégorie illustre la différence entre un contrôle syntaxique et un test de parcours. Tous les templates présents sont valides, mais le contrôleur peut demander un fichier absent. Un test qui appelle cette route avec une catégorie connue détecte un problème que le lint ne peut pas voir.

Le diagnostic du devis montre une autre différence : required décrit surtout une attente de saisie côté interface, alors qu'une contrainte serveur vérifie une donnée indépendamment du navigateur. Le test en mémoire reproduit précisément cette frontière sans enregistrer de données indésirables.

Une fiche de correction devrait conserver le symptôme, la cause, les fichiers modifiés, le résultat avant et après et le risque de régression. Les modifications applicatives ne font pas partie de la rédaction de ce dossier. Cette liste décrit donc le travail à réaliser et à revalider, sans prétendre que les problèmes ont déjà été corrigés.

Sources : contrôleurs, formulaires, vues et essais ; état local du 11 octobre 2026.

31 Procédure de préparation et de livraison

Une installation reproductible commence par une copie de la version choisie et des fichiers de verrouillage. Les variables nécessaires concernent notamment l'environnement Symfony, le secret applicatif, la connexion à MariaDB, le transport de courriel et SITE_MAINTENANCE. Leurs valeurs doivent être fournies par un canal distinct de la documentation publique.

Préparation locale

Dans un environnement neuf de développement, lancer les services Docker, installer les dépendances PHP depuis composer.lock et les dépendances JavaScript depuis package-lock.json. Créer une base dédiée, examiner les migrations puis les appliquer. Compiler les ressources et vérifier la présence des fichiers générés dans public/build ainsi que des manifests Encore.

Les commandes de contrôle utilisées dans ce projet sont php bin/console lint:twig templates, php bin/console lint:yaml config, php bin/console doctrine:schema:validate et php bin/phpunit. Dans les conteneurs, elles sont précédées de docker compose exec -T php. Le résultat de PHPUnit doit être lu : un code de sortie sans erreur ne suffit pas si aucun test n'a été exécuté.

Livraison préparée pour la production

Sur une copie destinée à la livraison, installer les dépendances nécessaires au fonctionnement avec une configuration de production, compiler les ressources avec npm run build et préparer le cache Symfony. La racine du serveur doit viser public, afin de ne pas exposer src, vendor, les fichiers d'environnement ou les documents du projet.

Avant toute migration de production, sauvegarder la base et les fichiers transférés, vérifier le SQL et prévoir une fenêtre de maintenance. Les cibles du Makefile qui suppriment la base ne doivent pas être utilisées pour cette étape. Le script de normalisation des statuts est un exemple où le retour arrière doit reposer sur une sauvegarde fiable.

Après livraison, vérifier la connexion, les autorisations, la lecture du catalogue, les formulaires, les ressources statiques, les logs et la réponse de maintenance. L'envoi de courriel doit être testé avec un destinataire de recette convenu. Ce chapitre propose une procédure adaptée au dépôt ; il ne constitue pas un journal attestant que toutes ces étapes ont été exécutées sur l'hébergement public.

La cible node du Dockerfile contient COPY php et npm build, alors que le projet est à la racine et que le script déclaré s'appelle build via npm run build. Cette cible n'est pas celle du service node-dev local ; elle doit être corrigée et testée avant de servir de chaîne de livraison.

Sources : Dockerfile ; compose.yaml ; Makefile ; webpack.config.js ; migrations ; procédure proposée.

32 Site publié et mode maintenance

Le domaine communiqué est https://www.laiguilledesvolcans.fr/. Le candidat indique que la page de maintenance est actuellement publique et que la connexion d'un administrateur par /login donne accès au site complet. Ce fonctionnement correspond à l'intention du MaintenanceSubscriber présent dans le dépôt.

Extrait du dépôt • src/EventSubscriber/MaintenanceSubscriber.php • lignes 32 à 46

```
32  if (!$event->isMainRequest() || !$this->maintenanceEnabled) {
33      return;
34  }
35
36  $path = $event->getRequest()->getPathInfo();
37
38  if (
39      $path === '/login'
40      || str_starts_with($path, '/build/')
41      || str_starts_with($path, '/assets/')
42      || str_starts_with($path, '/bundles/')
43      || $this->authorizationChecker->isGranted('ROLE_ADMIN')
44  ) {
45      return;
46  }
```

Le subscriber intervient sur kernel.request avec une priorité de -10. Lorsque la maintenance est active, il laisse passer /login, plusieurs chemins de ressources et les utilisateurs possédant ROLE_ADMIN. Pour les autres requêtes concernées, il renvoie public/maintenance.html avec le statut HTTP 503 et l'en-tête Cache-Control: no-store.

Le code 503 exprime une indisponibilité temporaire. L'en-tête évite que la réponse soit conservée comme une page habituelle. Un en-tête Retry-After pourrait être ajouté si une durée prévisible est connue. Le mode maintenance ne remplace pas la protection des routes administratives : ces contrôles doivent rester actifs après l'ouverture au public.

Périmètre de la preuve

La consultation du domaine par l'outil de recherche utilisé pour ce dossier n'a pas abouti. Le comportement de production est donc rapporté à partir de la déclaration du candidat et rapproché du code local, sans être présenté comme une mesure indépendante. Aucun identifiant administrateur n'a été demandé ni reproduit.

La maintenance applicative ne protège pas nécessairement les fichiers statiques servis directement par le serveur. Les logos placés sous public/uploads peuvent rester accessibles par leur URL, même si une page Symfony affiche une maintenance. Le stockage des pièces jointes doit être sécurisé indépendamment de ce subscriber.

Pour compléter la preuve de déploiement, conserver le type d'hébergement, la version PHP effective, la configuration du document root, le certificat HTTPS, la configuration de courriel, une capture datée et le résultat d'une recette avec un compte de démonstration. Ces éléments restent distincts des procédures locales Docker.

Sources : MaintenanceSubscriber ; public/maintenance.html ; déclaration du candidat ; production non vérifiée indépendamment.

33 Sauvegarde et continuité de service

Les données durables sont réparties entre MariaDB et les fichiers d'images ou de logos. Une sauvegarde de la base seule ne restaure pas les pièces jointes ; une copie du dossier uploads seule ne retrouve pas les relations, les coordonnées et les statuts. Le plan de sauvegarde doit donc traiter ces ensembles de façon cohérente.

Procédure proposée

Avant une mise à jour significative, effectuer une sauvegarde de la base et des fichiers, identifier la version du code et conserver un inventaire des migrations appliquées. Stocker la sauvegarde dans un emplacement à accès limité, distinct du répertoire public du site. La durée de conservation et la fréquence doivent être choisies avec la responsable de l'activité selon le volume et l'importance des demandes reçues.

Une restauration doit être testée dans un environnement séparé. Le test consiste à restaurer la base, remettre les fichiers au même emplacement logique, installer la version de code compatible et vérifier un échantillon de produits et de demandes fictives. Le simple fait qu'une commande d'export termine sans erreur ne prouve pas qu'une restauration est possible.

Le retour arrière doit également prendre en compte les migrations de données. La normalisation des statuts ne peut pas reconstituer automatiquement les valeurs initiales. Revenir seulement au code précédent après cette migration n'est donc pas un retour à l'état antérieur complet.

Exploitation et diagnostic

Le projet inclut Monolog pour les journaux et une page de maintenance pour interrompre temporairement l'accès public. Il reste à définir les personnes alertées, la procédure de consultation des logs et les informations à ne pas y enregistrer. Une surveillance pourrait vérifier les codes HTTP, les erreurs serveur et l'espace disque, sans collecter inutilement les messages ou les logos des visiteurs.

Aucun engagement chiffré de disponibilité, de perte maximale de données ou de délai de reprise n'a été fourni. Ce dossier n'invente donc pas de RPO, de RTO ou de contrat d'exploitation. Il documente les opérations à prévoir et les preuves à recueillir. La rotation des fichiers transférés, les images remplacées et les demandes arrivées à échéance doivent faire partie de cette politique de maintenance.

Une ouverture publique maîtrisée suppose enfin une version identifiée, des essais enregistrés et une possibilité de revenir à un état exploitable. Le retrait de la page de maintenance doit suivre ces vérifications, même si le site est déjà techniquement accessible au domaine définitif.

Sources : stockage ; migrations ; Monolog ; maintenance ; procédure proposée, sans essai de restauration attesté.

34 Veille technique et sécurité

La veille doit relier une source, une date, une observation et une décision concernant le projet. Le dépôt et les supports fournis ne contiennent pas de journal historique détaillé permettant de prouver une veille régulière pendant toute la formation. Les éléments ci-dessous correspondent à la revue documentaire du 11 octobre 2026 et à une méthode de suivi à poursuivre.

Sujet	Source de référence	Application au projet
Protection CSRF	Symfony 7.4	Distinguer Forms, connexion et contact manuel
Transfert de fichiers	OWASP File Upload	Revoir SVG, stockage et droits de lecture
Accessibilité	RGAA officiel	Langue, clavier, contrastes et formulaires
Données personnelles	CNIL	Information, conservation et pièces jointes
Certification	REAC, RE, France compétences	Relier les preuves aux compétences

La documentation Symfony rappelle que le composant Form gère le CSRF lorsqu'il est configuré, tandis qu'un formulaire HTML manuel demande un contrôle adapté. Cette différence s'applique au contact du projet. Elle conduit à proposer un formulaire Symfony ou une vérification explicite plutôt qu'à affirmer que toutes les pages bénéficient automatiquement de la même protection.

Les recommandations OWASP sur les fichiers conduisent à examiner la liste de formats autorisés, le nom généré et l'emplacement de stockage. Dans le projet, l'acceptation du SVG et le rangement public des logos méritent un traitement particulier. Il s'agit d'un risque de conception identifié, pas de l'annonce d'une vulnérabilité CVE propre à Symfony.

Pour les dépendances, un suivi futur peut s'appuyer sur les avis de sécurité des éditeurs et les outils d'audit de Composer et npm. Les résultats doivent être datés, rapprochés des versions verrouillées et évalués avant une mise à jour. Aucun bilan « zéro vulnérabilité » n'est annoncé : ces audits de dépendances n'ont pas été exécutés pour ce dossier.

Un journal utile conservera la source consultée, la version concernée, l'impact estimé, l'action décidée et le test après correction. Les décisions réellement prises pendant la formation pourront compléter cette page à partir des notes personnelles du candidat.

Sources : Symfony CSRF ; OWASP File Upload ; RGAA ; CNIL ; consultation du 11 octobre 2026.

35 Bilan et perspectives

Le projet relie une expression de besoin concrète à des interfaces publiques, un modèle relationnel et un espace de gestion. Les réalisations les plus significatives sont le catalogue catégorisé, la demande de devis, les avis modérés et les contrôles d'accès. Elles permettent d'expliquer le rôle de Symfony, Twig, Doctrine et JavaScript sur un même parcours.

Les difficultés mentionnées dans ma présentation concernent l'architecture Symfony avec Docker, les associations Doctrine, la circulation des données entre contrôleur et vue et l'intégration des maquettes. L'analyse du dépôt permet de les illustrer par des cas précis : deux routes de catégorie ont divergé, une contrainte visuelle ne valide pas les données côté serveur et le stockage de fichier doit être coordonné avec l'écriture SQL.

Priorités avant la soutenance

Les premières corrections concernent la validation serveur du devis, la route en erreur, la visibilité des produits inactifs et le traitement des logos. Elles doivent être accompagnées d'essais avant et après. Le layout doit ensuite être complété et les parcours mobiles et accessibles vérifiés. Une suite de tests ciblée sur les règles métier apportera une preuve durable au lieu de dépendre uniquement d'une démonstration manuelle.

La publication existe sous maintenance, mais la recette de production, la sauvegarde et les caractéristiques de l'hébergement doivent être documentées. La traduction, la recherche reliée à l'interface, la pagination et les notifications peuvent suivre après la stabilisation du périmètre central.

Conclusion

Ce dossier démontre les choix et les mécanismes présents dans la version examinée, tout en conservant les écarts visibles. Il constitue une base de présentation technique à relire et à actualiser après les correctifs. Les résultats observés ne permettent pas encore d'affirmer une conformité globale de sécurité, d'accessibilité ou de recette fonctionnelle.

Technical summary: L'Aiguille des Volcans is a server-rendered Symfony application for an embroidery workshop. Visitors browse products and submit quote requests. Signed-in users can submit reviews, and administrators manage the catalogue and moderation. Doctrine stores business data in MariaDB. The current review identified missing server-side validation and an unavailable category template. Deployment documentation and repeatable functional tests are the next priorities.

Sources : présentation du candidat ; code et essais ; synthèse en anglais rédigée pour le dossier.

36 Sources et pièces de preuve

Les références ci-dessous permettent de retrouver l'origine des descriptions et de distinguer les exigences du titre, les supports de conception et les observations techniques. Les pages du RE et du REAC correspondent à leur pagination interne ; le document combiné comporte aussi sa propre pagination PDF.

Référentiels et documents fournis

[R1] EDUCENTRE_RNCP31114_REAC.pdf : REAC DWWM TP-01280, millésime 04, version du 24 mai 2023 ; vue synoptique p. 8, fiches compétences p. 15 à 30.

[R2] Référentiel_Activités_Compétences_Evaluation_TP_DWWM (1).pdf : REAC et RE 2023 ; RE p. 5-6 pour le volume et le contenu du dossier, p. 9 et suivantes pour les critères d'évaluation.

[R3] RNCP37674 - TP - Développeur web et web mobile.pdf et fiche officielle https://www.francecompetences.fr/recherche/rncp/37674/ ; consultée le 11 octobre 2026.

[R4] Corentin_Polard_Dossier_Projet.pdf : référence de structure et de niveau de détail. Aucun fait, résultat ou réalisation de ce candidat n'est attribué au présent projet.

[R5] Aureo_Ruffier_Presentation_Projet.pptx : support en cours, utilisé pour le contexte, les thèmes techniques et les difficultés déclarées. Il n'a pas été modifié pour cette rédaction.

Sources techniques du projet

[P1] Dépôt projet_broderie, référence b406ce8, état observé le 11 octobre 2026 ; src, templates, assets, config, migrations, Dockerfile, compose.yaml, Makefile et fichiers de dépendances.

[P2] output/docs/cahier_des_charges.md ; schémas et exports historiques de output/docs ; maquettes et wireframes de output/design. Les supports mobiles du paquet nouveaux-svg sont des adaptations documentées dans son LEIA-ME.

[P3] Contrôles du 11 octobre 2026 : syntaxe Twig et YAML, schéma Doctrine, exécution PHPUnit, huit requêtes HTTP locales, cinq soumissions de formulaire et quatre valeurs de statut. Les captures montrent des rendus locaux, sans connexion à l'administration de production.

Documentation externe

[D1] https://symfony.com/doc/7.4/security/csrf.html ; [D2] https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html ; [D3] https://accessibilite.numerique.gouv.fr/ ; [D4] https://www.cnil.fr/fr/comprendre-le-rgpd/les-six-grands-principes-du-rgpd. Ces références servent de cadre technique ; elles ne certifient pas le projet.

Les informations personnelles de couverture, le contexte individuel et l'état de publication ont été communiqués par le candidat. Les preuves restant à joindre sont les caractéristiques de l'hébergement, la recette de production, les décisions client et l'historique personnel de veille.

Annexe A Modèle physique des relations

Ce schéma synthétique représente les clés et les relations de la version examinée. Les attributs détaillés figurent au chapitre 17. PK désigne une clé primaire, FK une clé étrangère et UQ une contrainte unique. Les attributs de compte et de devis contenant des données personnelles ne sont pas illustrés par des valeurs réelles.

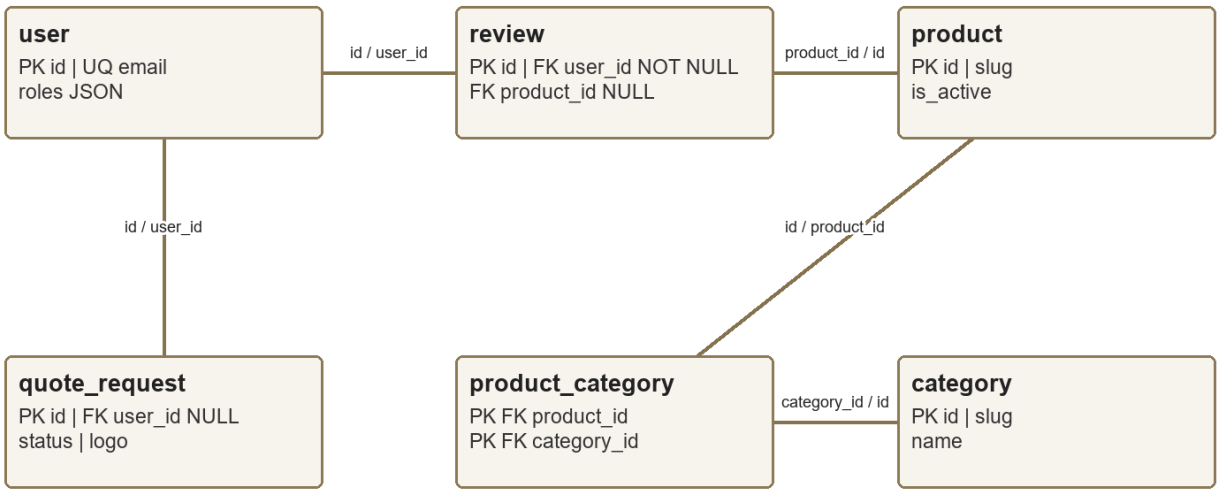


Figure A1 Schéma physique simplifié reconstitué à partir du mappage Doctrine

Les relations de product_category utilisent une clé primaire composée. review.user_id est obligatoire, tandis que review.product_id et quote_request.user_id peuvent être NULL. Le lien produit vers avis prévoit ON DELETE CASCADE. La table technique messenger_messages est volontairement séparée du modèle métier.

Le contrôle doctrine:schema:validate a confirmé le mappage et la synchronisation de la base locale. Il ne garantit pas l'absence de perte dans toutes les migrations historiques.

Annexe B Maquette web du devis

Support graphique déjà fourni dans le projet, conservé pour la comparaison avec le formulaire et son adaptation mobile. Cette figure ne constitue pas une capture de la production actuelle.

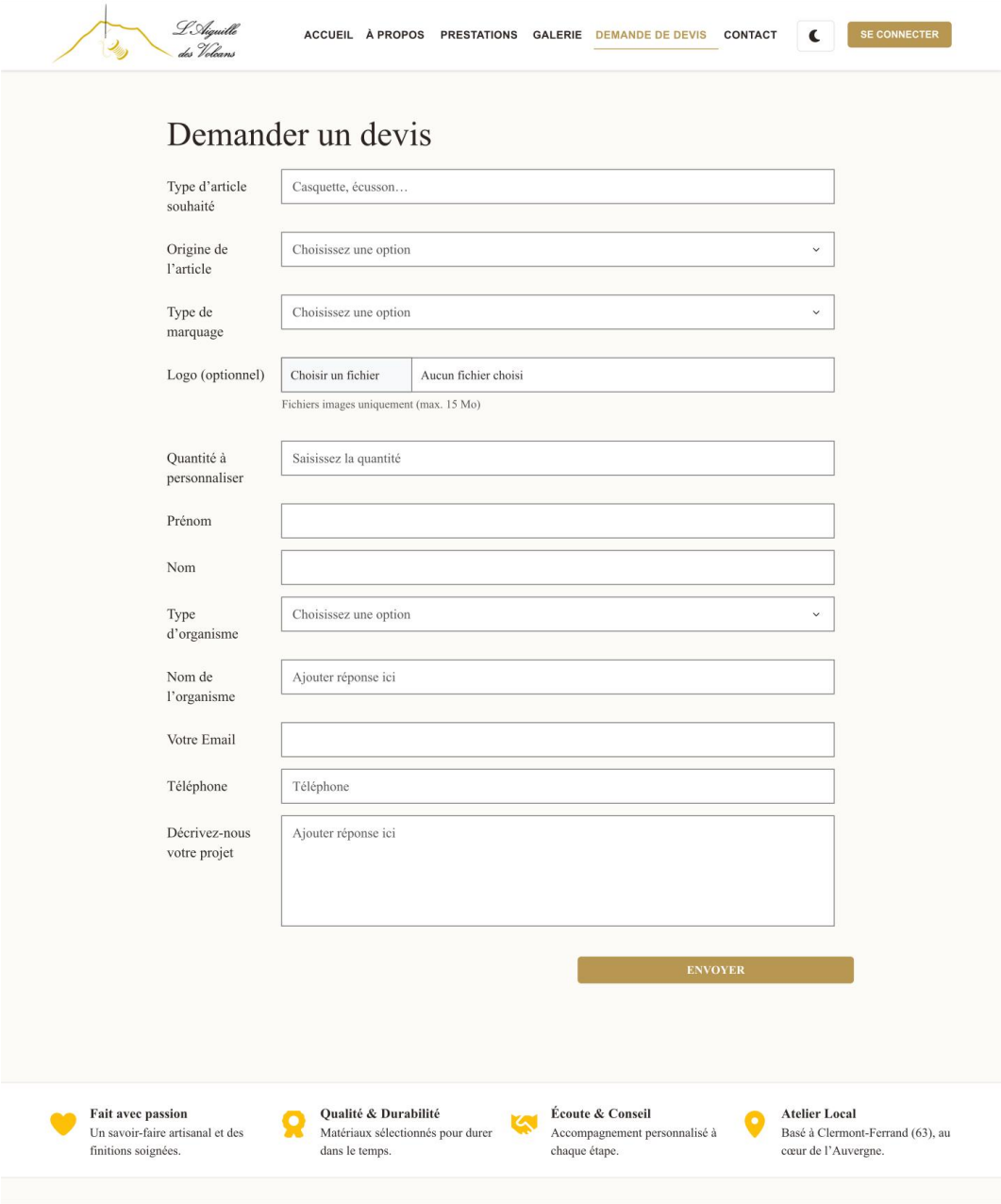


Figure B1 Maquette web du devis issue du support SVG fourni

Annexe C Extraits du formulaire de devis

Les extraits ci-dessous montrent la séparation entre le fichier temporaire non mappé et la quantité persistante. Ils sont reproduits depuis le dépôt et doivent être lus avec les résultats des essais du chapitre 29.

Extrait du dépôt • src/Form/QuoteRequestType.php • lignes 48 à 61

```
48  ->add('logoFile', FileType::class, [
49      'label' => 'Logo (optionnel)',
50      'mapped' => false,
51      'required' => false,
52      'help' => 'Fichiers images uniquement (max. 15 Mo)',
53      'attr' => ['accept' => 'image/*'],
54      'constraints' => [
55          new File([
56              'maxSize' => '15M',
57              'mimeTypes' => ['image/jpeg', 'image/png', 'image/gif', 'image/webp',
    'image/svg+xml'],
58              'mimeTypesMessage' => 'Veuillez sélectionner une image valide.',
59          ]),
60      ],
61  ])
```

Extrait du dépôt • src/Form/QuoteRequestType.php • lignes 62 à 77

```
62  ->add('quantity', IntegerType::class, [
63      'label' => 'Quantité à personnaliser',
64      'required' => true,
65      'attr' => ['placeholder' => 'Saisissez la quantité', 'min' => 1],
66      'constraints' => [new GreaterThanOrEqual(1)],
67  ])
68  ->add('firstName', TextType::class, [
69      'label' => 'Prénom',
70      'mapped' => false,
71      'required' => true,
72  ])
73  ->add('lastName', TextType::class, [
74      'label' => 'Nom',
75      'mapped' => false,
76      'required' => true,
77  ])
```

Le type File vérifie la taille et une liste de MIME. Il ne rend pas automatiquement le SVG inoffensif. GreaterThanOrEqual contrôle la quantité côté serveur ; required ne remplace pas une contrainte NotBlank sur les champs textuels. Les corrections proposées doivent rester distinctes de ces extraits de la version examinée.

Annexe D Maquette mobile du devis

Adaptation mobile fournie dans les supports graphiques du projet. Les deux parties se lisent de gauche à droite. Elles présentent une intention de conception et ne remplacent pas une recette sur téléphone.

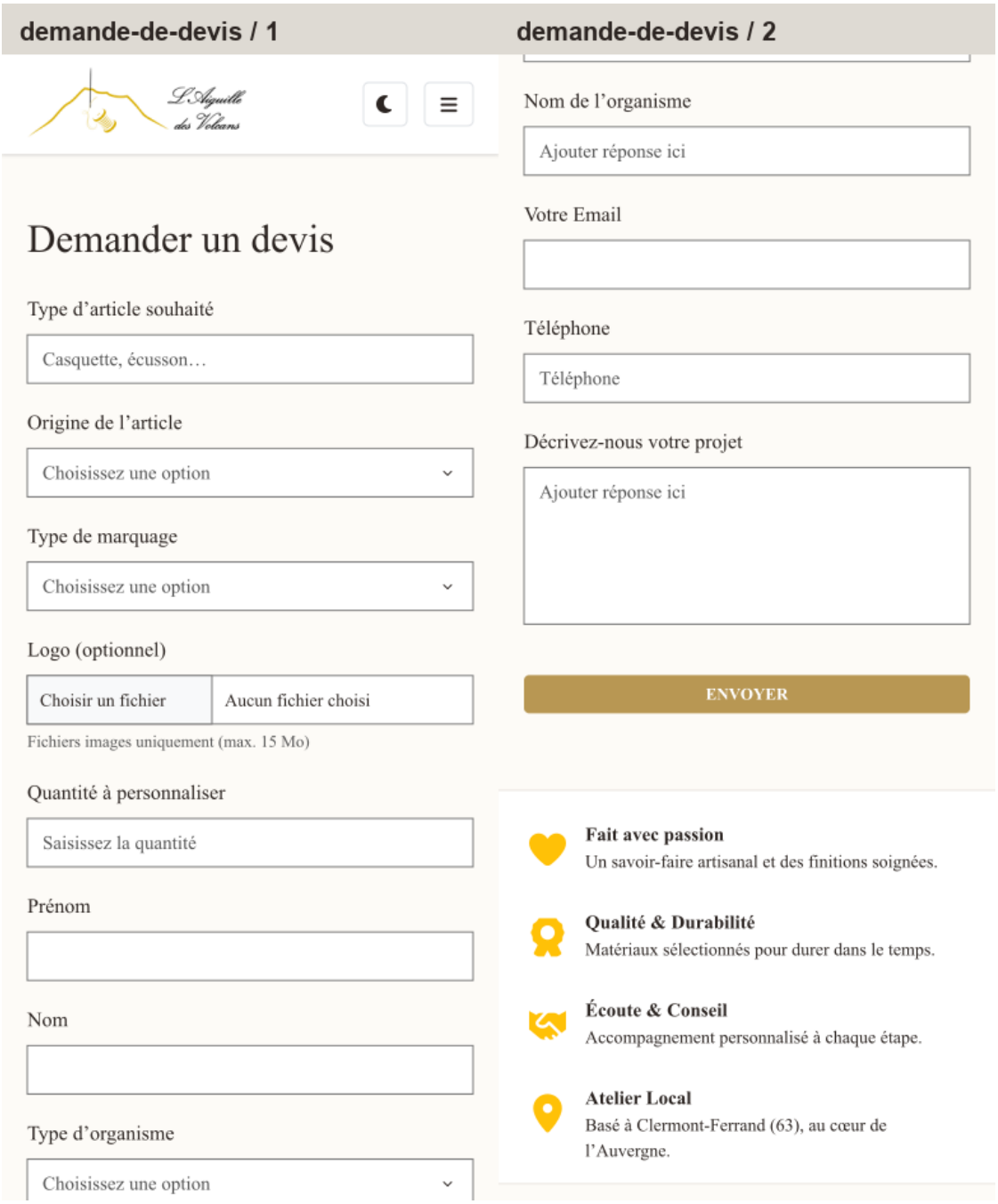


Figure D1 Maquette mobile complète du formulaire en deux parties